Naive Bayes

Gaussian Naive Bayes from scratch

The problem

- Classify using probabilities
- Few training examples, many features
- Need something fast and simple

Bayes' rule + the naive assumption

$$P(c \mid x) \propto P(c) \prod_{j} P(x_j \mid c)$$

- Assume features independent given the class
- Then each feature gets its own simple model

Gaussian likelihood in log space

$$\log P(x_j \mid c) = -\tfrac{1}{2}\left[\log 2\pi\sigma_{cj}^2 + \frac{(x_j - \mu_{cj})^2}{\sigma_{cj}^2}\right]$$

- Products of tiny numbers underflow → sum logs
- Add a tiny ε to variances

GaussianNB

```
class GaussianNB:
    def fit(self, X, y):
        X, y = np.asarray(X, float), np.asarray(y)
        self.classes_ = np.unique(y)
        self.mu = np.array([X[y == c].mean(0) for c in self.classes_])
        self.var = np.array([X[y == c].var(0) + 1e-9 for c in self.classes_])
        self.logprior = np.log(np.array([np.mean(y == c) for c in self.classes_]))
        return self

    def predict_log_proba_unnormalized(self, X):
        X = np.asarray(X, float)
        ll = -0.5 * (np.log(2 * np.pi * self.var)[None] + (X[:, None, :] - self.mu[None]) ** 2 / self.var[None]).sum(-1)
        return ll + self.logprior

    def predict(self, X):
        return self.classes_[np.argmax(self.predict_log_proba_unnormalized(X), axis=1)]
```

Complexity & trade-offs

- Train O(n·d), predict O(c·d)
- Works with tiny datasets; no tuning
- Independence rarely true → probabilities poorly calibrated
- Ranking of classes is often still good

Recap

- Posterior $\propto$ prior $\times$ product of likelihoods
- Gaussian per feature per class
- Work in log space
- Fast, strong baseline